

TALLER PRACTICO - PREGUNTAS DE REFLEXION

Internacionalizacion (i18n) + Docker + OTP

Aplicado al Sistema de Gestion de Vacunacion y SaludBoyaca

Julieth Dayana Peña Barrera

Ficha: 3171062

Instructor: Osman Alonso Aranguren Escobar

SERVICIO NACIONAL DE APRENDIZAJE - SENA

Regional Boyaca - Centro Industrial de Mantenimiento y Manufactura

Tecnologia en Analisis y Desarrollo de Software - ADSO

Sogamoso, Boyaca - 2026

11. Preguntas de Reflexión

11.1 Sobre el Código de Referencia y Arquitectura

- Estudiaste el LoginServlet del sistema de vacunación y luego creaste el LoginServlet de SaludBoyaca. Cuántas líneas de código son iguales o muy similares? Cuáles tuviste que cambiar completamente? Qué te dice esto sobre el valor de estudiar código bien estructurado antes de escribir el propio?

Al comparar ambos Servlets, diría que un 60% de las líneas quedaron similares en SaludBoyaca. La estructura general del método doPost(), la forma de obtener los parámetros con request.getParameter(), la validación de credenciales a través del DAO, el manejo de la sesión con HttpSession y las redirecciones con sendRedirect() siguen exactamente el mismo patrón en ambos proyectos.

Lo que realmente si tuve que cambiar completamente fue el bloque del CAPTCHA. En el sistema de vacunación, doGet() generaba la imagen del CAPTCHA y la guardaba en sesión como 'captchaText', y doPost() la validaba antes de continuar. En SaludBoyaca ese bloque desaparece por completo. En su lugar, después de validar las credenciales con el DAO, el código genera el OTP con OTPService.generarOTP(), guarda el código y el timestamp en sesión, obtiene los textos del correo desde el ResourceBundle según el idioma activo y llama a OTPService.enviarOTP(). Esas líneas son completamente nuevas y específicas del dominio de SaludBoyaca.

Lo que me queda claro es que estudiar primero el código de referencia me ahorra bastante tiempo. Entender cómo funcionaba la sesión, cómo se manejaban los roles y cómo se integraba la i18n me permitió saber exactamente qué partes podía adaptar y cuáles tenía que construir desde cero. Si hubiera empezado a escribir sin revisar el código de vacunación, habría terminado resolviendo problemas que ya estaban resueltos.

- El patrón switch/acción de PacienteServlet.java se repite casi idéntico en CitaServlet.java. Por qué los buenos sistemas tienen este tipo de patrones repetibles? Qué problema causaría si cada Servlet fuera completamente diferente en estructura?

El patrón switch/acción hace que todos los Servlets CRUD hablen el mismo idioma. Cuando llega una petición a CitaServlet, el parámetro 'acción' le dice al switch si debe listar, insertar, editar o eliminar, y cada caso delega al DAO. Lo mismo hace PacienteServlet, y lo mismo haría cualquier otro Servlet del sistema. Eso significa que si alguien entra al proyecto sin haberlo construido, puede entender cualquier Servlet en segundos porque la lógica es la misma.

Si cada Servlet tuviera una estructura completamente diferente, el sistema sería difícil de mantener porque cada modificación exigiría entender de nuevo la lógica particular de ese componente. La incorporación de un nuevo desarrollador al equipo sería más costosa, las pruebas serían más complejas y la posibilidad de introducir errores aumentaría. La uniformidad no es solo estética, es lo que hace que el código sea mantenible a largo plazo.

- Compara el esquema de la BD de vacunación (5 tablas) con el de SaludBoyaca (7 tablas). ¿Qué decisiones de diseño tomaste diferentes y por qué? ¿Qué aprendiste del esquema de referencia que aplicaste directamente?

El esquema de vacunación es más sencillo porque el dominio lo permite: pacientes, vacunas, registros, usuarios y log. En SaludBoyaca el dominio es más complejo y eso se refleja en las dos tablas adicionales. La tabla especialidades existe porque no tenía sentido guardar la especialidad como un texto libre en usuarios o en citas, normalizarla permite agregar o renombrar especialidades sin tocar otras tablas. La tabla horarios modela la disponibilidad real de cada médico por día y franja horaria, que es información que el sistema necesita para validar si una cita es posible antes de guardarla.

La tabla otp_tokens fue una decisión consciente de seguridad. El sistema de vacunación en el taller guarda el OTP solo en sesión, lo que funciona para el ejercicio, pero en SaludBoyaca decidí persistirlo en BD para poder marcarlo como usado y evitar que se reutilice. Eso resuelve un escenario real de ataque que con solo la sesión no se puede controlar.

Del esquema de referencia apliqué directamente el uso de ENUM para campos controlados como el rol y el estado de las citas, las claves foráneas con sus índices, y los campos de timestamp automático con DEFAULT CURRENT_TIMESTAMP. Son convenciones que el esquema de vacunación ya tenía bien implementadas y no había razón para cambiarlas.

11.2 Sobre Internacionalización y OTP

- LocaleFilter.java del sistema de vacunación fue reutilizado casi sin cambios en SaludBoyaca. ¿Qué característica del diseño de esa clase la hace reutilizable? ¿Qué habría que cambiar si quisieras agregar un idioma nuevo como portugués a tu aplicación?

LocaleFilter.java es reutilizable porque no sabe nada del dominio. No tiene referencias a vacunas, citas ni pacientes. Solo le importa el parámetro lang que llega en la URL, lo valida contra una lista de idiomas permitidos y lo guarda en sesión. Al no estar acoplado a ninguna lógica de negocio, se puede llevar de un proyecto a otro sin problema.

Para adaptarlo a SaludBoyaca solo cambie la lista de idiomas válidos de {es, en, fr, it} a {es, en, it}, eliminando el francés. El idioma por defecto sigue siendo 'es'. El resto del código lo deje igual porque el mecanismo de detección y almacenamiento en sesión no depende del dominio.

Si se quisiera agregar portugués a la aplicación, los pasos serían: primero, agregar 'pt' a la lista de idiomas válidos en LocaleFilter.java; segundo, crear el archivo messages_pt.properties con todas las claves traducidas al portugués; y tercero, agregar el enlace con la bandera de Brasil o Portugal al selector de idiomas en templates/header.jsp. No sería necesario modificar ningún Servlet ni DAO porque la i18n está completamente desacoplada de la lógica de negocio.

- El CAPTCHA grafico protege contra bots automatizados. El OTP por correo tambien lo hace, pero ademas verifica la identidad. Por que el OTP reemplaza al CAPTCHA en el login y no se usan juntos? En que modulo de SaludBoyaca sigue siendo necesario el CAPTCHA y por que no se puede reemplazar alli?

El OTP reemplaza al CAPTCHA en el login porque cumple una funcion superior. El CAPTCHA solo verifica que quien intenta acceder es un humano, pero no sabe si es el dueño legitimo de la cuenta. El OTP, en cambio, verifica ademas que el usuario tiene acceso al correo electronico registrado en la base de datos. Un bot no puede leer ese correo, asi que el OTP ya cumple la funcion del CAPTCHA y encima agrega verificacion de identidad real. Poner los dos juntos en el login no sumaria seguridad, solo le daria mas pasos innecesarios al usuario.

El modulo donde el CAPTCHA si tiene que quedarse es la consulta publica, el /consulta-cita de SaludBoyaca. Ahi el ciudadano busca sus citas con el numero de documento sin tener cuenta en el sistema. Como no hay cuenta registrada, no hay correo al cual enviar un OTP. El OTP es tecnicamente imposible en ese contexto. El CAPTCHA es la unica defensa anti-bot disponible ahi, y por eso se conserva.

- El codigo OTP se genera con SecureRandom. Por que se usa SecureRandom en lugar de la clase Random normal de Java? Si guardas el OTP solo en sesion HTTP vs en la tabla otp_tokens de la BD, cual es mas seguro y por que? Que escenario de ataque resuelve guardarlo en la BD?

Random en Java usa un algoritmo que, si se conoce un valor generado, se puede predecir los siguientes. Para un password de un solo uso eso es un problema grave. SecureRandom usa fuentes de entropia del sistema operativo, como tiempos de hardware o /dev/urandom en Linux, y produce numeros que son impredecibles criptograficamente. Para algo de seguridad como el OTP no hay otra opcion.

Guardar el OTP solo en sesion funciona para el ejercicio del taller, pero tiene limitaciones reales. Si Tomcat se reinicia, la sesion desaparece y el usuario queda bloqueado. Si el usuario intenta verificar desde otro navegador o dispositivo, la sesion no existe ahi y el codigo tampoco. Guardarlo en la tabla otp_tokens de la BD resuelve esos casos porque el codigo existe de forma independiente a la sesion.

El escenario de ataque que resuelve persistirlo en BD es la reutilizacion de codigos. Si el OTP solo vive en sesion y se borra al usarse, no queda registro de que fue consumido. Si alguien intercepta el codigo, podria intentar usarlo en otra sesion antes de que expire. Con la tabla otp_tokens y el campo 'usado = 1', el sistema rechaza cualquier segundo intento con ese mismo codigo, aunque no haya expirado. Ademas permite auditar intentos fallidos con IP y timestamp, algo que la sesion jamas podria dar.

11.3 Sobre Docker, Despliegue y Decisiones de Produccion

- El Dockerfile de SaludBoyaca usa Multi-Stage Build. Si no lo usaras y usaras una sola etapa con maven:3.8.6-openjdk-14, cuanto pesaria la imagen? Por que ese tamaño importa en Back4App (256 MB de RAM) o en Koyeb (0.1 vCPU)?

La imagen maven:3.8.6-openjdk-14 sola ya pesa entre 500 y 600 MB. Si se usa esa como imagen final sin Multi-Stage Build, el resultado incluye el JDK completo, Maven con su repositorio local de dependencias descargadas durante el proceso de compilacion, y todos los archivos fuente del proyecto. El resultado seria una imagen que facilmente superaria los 800 MB.

Con Multi-Stage Build la cosa cambia. La primera etapa usa maven para compilar y generar el WAR, pero la imagen final se construye sobre tomcat:9.0-jdk14-openjdk que es mucho mas liviana porque solo tiene el JRE y Tomcat, sin nada del proceso de compilacion. El resultado queda en unos 250 a 350 MB.

En Back4App eso importa directamente porque el plan gratuito da solo 256 MB de RAM. Con una imagen pesada Tomcat no arranca y el contenedor muere con OutOfMemoryError. Por eso ademas hay que ajustar la JVM con `JAVA_TOOL_OPTIONS=-Xms64m -Xmx200m` para que Tomcat no se coma toda la memoria. En Koyeb con 0.1 vCPU, una imagen mas liviana reduce el tiempo de arranque del contenedor, que con un CPU tan limitado puede marcar la diferencia entre un despliegue exitoso y uno que timeout.

- Desplegaste SaludBoyaca en 2 hosting. Describe exactamente que pasos seguiste, que errores encontraste y como los resolviste. Cual fue el error mas dificil de depurar en produccion? Como ves los logs de un contenedor en Render o Koyeb?

- Pasos seguidos:

Primero subí el proyecto a GitHub. Inicialicé el repositorio con `git init` dentro de la carpeta del proyecto, luego `git add .` y `git commit -m "primer commit"`. Antes de hacer push tuve que resolver un problema de ownership con `git config --global --add safe.directory E:/SaludBoyaca`. También creé un `.gitignore` para excluir la carpeta `target/` que contiene archivos compilados que no deben subirse. Finalmente conecté el repositorio remoto con `git remote add origin` y ejecuté `git push`.

Para la base de datos, creé una instancia PostgreSQL gratuita en Render, luego conecté esa base de datos desde pgAdmin 4 usando el hostname, puerto, usuario y contraseña que provee Render. Desde el Query Tool de pgAdmin ejecuté el script SQL con todas las tablas y datos iniciales.

Para el primer hosting en Render creé un Web Service conectado al repositorio de GitHub. Configuré las variables de entorno `DB_URL`, `DB_USER` y `DB_PASS` apuntando a la base de datos PostgreSQL de Render. Render detectó el Dockerfile automáticamente y desplegó la app. Para el segundo hosting en Back4App hice lo mismo: importé el repositorio de GitHub,

configuré las mismas variables de entorno y Back4App construyó la imagen Docker y desplegó la app.

Para el envío de OTP por correo, el sistema usaba SMTP (puerto 587 y 465) pero Render bloquea esos puertos en el plan gratuito. La solución fue migrar a Brevo, que envía correos a través de su API por HTTPS (puerto 443), que no está bloqueado. Creé una cuenta en Brevo, generé una API Key y modifiqué OTPService.java para usar la API REST de Brevo en lugar de JavaMail. La clave API se configuró como variable de entorno BREVO_API_KEY en ambos hostings.

- Errores encontrados y cómo se resolvieron:

El primer error fue detected dubious ownership, resuelto con `git config --global --add safe.directory`.

El segundo fue que GitHub bloqueó el push porque detectó la API Key de Brevo hardcodeada en el código. Se resolvió eliminando la key del código, dejando solo `System.getenv("BREVO_API_KEY")`, y haciendo `git reset --soft HEAD~2` para reescribir el historial antes de hacer `git push --force`.

El tercero fue un error de compilación en LoginServlet.java porque el método enviarOTP cambió de lanzar MessagingException a Exception, y el servlet no lo manejaba correctamente.

- El error más difícil de depurar en producción:

El más difícil fue el bloqueo de SMTP. La app funcionaba perfectamente en local pero en producción el OTP nunca llegaba. Los logs mostraban `Couldn't connect to host, port: smtp.gmail.com, 587; timeout -1` y luego al cambiar al puerto 465 el mismo error. No era un problema de credenciales sino una restricción de red del hosting gratuito que bloquea todas las conexiones SMTP salientes. La solución fue cambiar completamente el mecanismo de envío a una API HTTP.

- Cómo ver los logs en Render:

En Render se accede al dashboard, se selecciona el Web Service y se hace clic en la pestaña Logs. Ahí aparecen en tiempo real los logs del contenedor Docker, incluyendo la salida de Tomcat y cualquier error de la aplicación.

En Back4App se accede a la app, en el panel izquierdo hay una sección de Deployments donde se puede ver el log de construcción y ejecución del contenedor en tiempo real.

- Eres el desarrollador de SaludBoyaca y el Centro de Salud de Paipa te dice que van a tener 50 usuarios simultaneos. Evaluando los 5 hosting del taller, cual recomendarias para produccion real? Cual seria tu segunda opcion? Justifica con los datos de tu tabla comparativa.

Para un sistema real en el Centro de Salud de Paipa con 50 usuarios simultaneos, la primera opcion es Oracle Cloud Always Free. Es el unico del taller que da recursos de verdad sin cobrar: hasta 4 OCPU y 24 GB de RAM en el plan permanente. Con eso Tomcat tiene suficiente margen para manejar carga real sin sudar. Al ser una VM dedicada tienes control total sobre el servidor, los logs, la configuracion de Tomcat y la JVM. Y lo mas importante para un sistema de salud: nunca se duerme.

La desventaja es que la configuracion es mas compleja. Hay que manejar Linux, configurar reglas de firewall para el puerto 8080, instalar Docker manualmente y gestionar el SSL a mano porque Oracle no lo da automatico como los otros. Pero para produccion real esa inversion vale la pena.

La segunda opcion seria Koyeb. A diferencia de Render, Koyeb no hiberna el servicio tras 15 minutos de inactividad, lo que es clave para un sistema donde el personal clinico puede conectarse en cualquier momento del dia. Tiene 512 MB de RAM, SSL automatico, dominio gratuito y el despliegue es sencillo. Para un equipo que no quiere administrar servidores, Koyeb es la mejor opcion entre los cinco.

Render no seria buena eleccion para este caso porque la hibernacion de 15 minutos significaria que la primera consulta del dia tardaria 30 segundos en responder, lo cual no es aceptable en un entorno clinico. Y Back4App con 256 MB de RAM no da para 50 usuarios bajo carga real de Tomcat.